

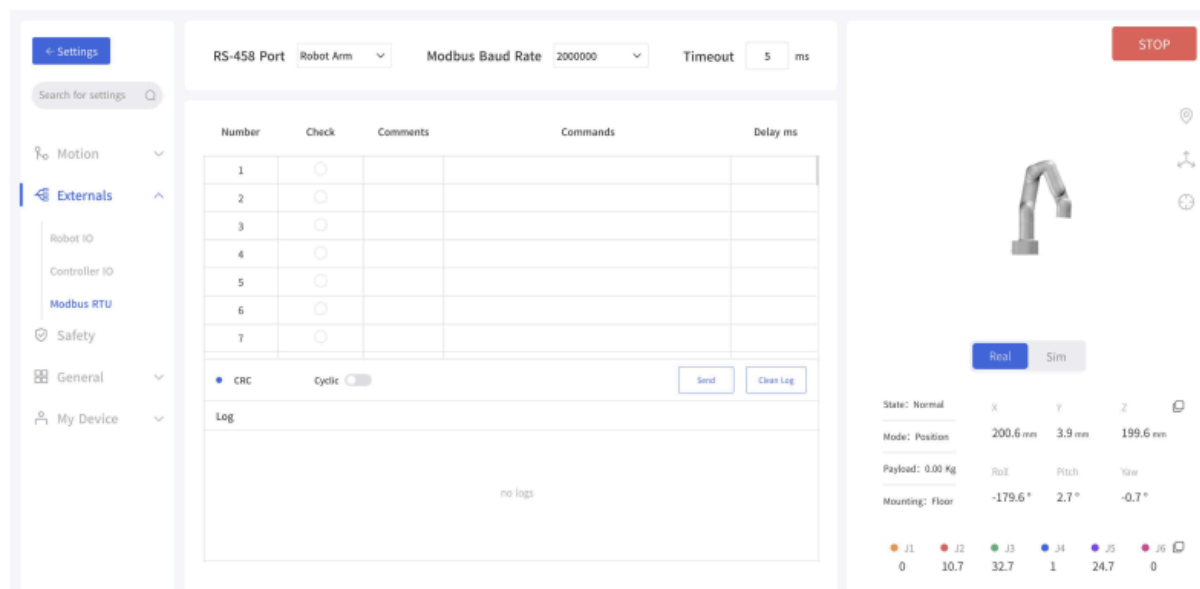
Slanje komandi iz web framework-a

Za paljenje i upravljanje same ruke postoji sajt. Laptop smo povezali na Ethernet ruter, namestili statički ip i u browseru otvorili link: <http://192.168.1.163:18333/>

Podešavanje statičkog ip-a:

ipv4
ip: 192.168.1.100
netmask: 255.255.255.0
gateway: prazno

Klikom na *Externals*, pa na *Modbus RTU*, otvorili smo:



Povezali smo RS485 dongle (A -> M_A, B -> M_B) na Control-box i laptop.
rs485 dongle

Izabrali smo Control-box kao *RS-485 Port*, postavili *Modbus Baud Rate* na 115200. Kliknuli *Check* polje i kao komandu upisali 09 10 03 E8 00 03 06 00 00 00 00 00 00 00 73 30.

Povezali smo i osciloskop kako bismo mogli da vidimo šta dobijamo od podataka. Klikom na dugme *Send*, podaci su se poslali, na osciloskopu (nakon što smo omogućili prikaz oba kanala i konkretnih bajtova), videli smo podatke.

Slanje komandi iz desktop aplikacije

Nakon uspostavljanja osnovne mrežne konekcije, cilj je bio verifikovati da kontrolna kutija robotske ruke Lite 6 ispravno interpretira Modbus komande. Za ovaj korak korišćen je softver ModbusTools (mbclient) na Linux operativnom sistemu, koji smo pronašli na Github-u.

1. Pokretanje klijenta

Prvi korak bio je pokretanje testnog klijenta iz terminala:

```
berenic@berenic-ThinkBook-16-G6-ABP:~/Documents/ModbusTools$ ./build/mbclient-0.5.0
```

2. Konfiguracija mrežnog porta (Connection Setup)

Kreiran je novi komunikacioni port sa sledećim parametrima:

Name: Lite6_Robot

Type: TCP (Modbus TCP protokol)

Host: 192.168.1.163

Port: 502

Timeout: 3000 ms

3. Definisanje uređaja (Device/Slave Setup)

Unutar porta, definisan je konkretan uređaj:

Name: Arm_Control

Unit (Slave ID): 1

PRETPOSTAVKA: ID 1 je za kontrolnu kutiju, a ID 9 za šaku.

4. Slanje testne poruke

Putem opcije Tools > Send Message, odabrana je funkcija:

Function: 03 - Read Holding Register

Address: 1

Count: 1

Objašnjenje: Funkcija 03 služi za čitanje podataka iz memorije robota. Registar na adresi 0x1 je Get version information. Ovo je najsigurnija komanda za test jer samo čita podatak bez pomeranja fizičkih delova ruke. Opcija Count nam daje mogućnost da izaberemo koliko uzastopnih registra želimo da pročitamo.

5. Rezultat i analiza logova

Klikom na Send, u logu se pojavilo:

- Tx: 00 03 00 00 00 06 01 03 00 00 00 01
 - Prva dva bajta: Transaction ID
 - Druga dva: Protokol ID
 - Treća dva: Dužina ostatka poruke
 - Sledeći 1 bajt: Adresa registra
 - Ostalo: Parametri (naredme komande)

- Rx: 00 03 00 00 00 05 01 03 02 00 02
 - Prva dva bajta: Transaction ID
 - Druga dva: Protokol ID
 - Treća dva: Dužina ostatka poruke
 - Sledeći 1 bajt: Adresa registra
 - Sledeći 1: Statusni bit
 - Ostalo: Parametri (naredne komande)

Slanje komandi direktno iz ROS2 koda

Nakon što smo od Subotića dobili testnu rutinu `glue_events_routines`, bilo je potrebno da je dodamo potrebne i uklonimo nepotrebne biblioteke i ispravimo greške.

Nakon toga smo tražili Modbus servis, gde se ispostavilo da već postoji ugrađen unutar postojećeg API-ja.

U SDK fajlovima smo morali da omogućimo i konfigurišemo Modbus funkcionalnosti vezane za TGPIO RS-485 komunikaciju, čime je omogućeno korišćenje servisa `getset_tgpio_modbus_data` za direktno slanje Modbus poruka iz ROS2 aplikacije.

U njegov kod, dodali smo sledeće linije, jer smo hteli da testiramo da li servis radi:

```
rclcpp::Client<xarm_msgs::srv::GetSetModbusData>::SharedPtr modbus_client;

modbus_client
node->create_client<xarm_msgs::srv::GetSetModbusData>("/ufactory/getset_tgpio_modbus_data")
;
    modbus_client->wait_for_service(chrono::seconds(3));

if (cmds.at("rs485") != "") {
    auto modbus_req = make_shared<xarm_msgs::srv::GetSetModbusData::Request>();
    modbus_req->host_id = 10;
    modbus_req->is_transparent_transmission = false;
    modbus_req->use_503_port = false;
    modbus_req->modbus_data = {
        0x00, 0x01,    // Transaction ID
        0x00, 0x02,    // Protocol
        0x00, 0x08,    // Length (samo register, bez parametara)
        0x7F,          // Register: Get error and warning code
        0x09,
        0x0A, 0x15,
        0x00, 0x80, 0x80, 0x43
    };
    modbus_req->modbus_length = 14;
    modbus_req->ret_length = 8;

    modbus_client->async_send_request(
        modbus_req, [](rclcpp::Client<xarm_msgs::srv::GetSetModbusData>::SharedPtr
future) {
        auto response = future.get();

        RCLCPP_INFO_STREAM(
            node->get_logger(),
            "RS485 ret = " << (int)response->ret
        );

        RCLCPP_INFO_STREAM(
            node->get_logger(),
            "RS485 message = " << response->message
        );

        RCLCPP_INFO_STREAM(
            node->get_logger(),
```

```

        "RS485 ret_data size = " << response->ret_data.size()
    );

    std::stringstream ss;

    for (auto b : response->ret_data) {
        ss << "0x"
        << std::hex
        << std::setw(2)
        << std::setfill('0')
        << (int)b
        << " ";
    }

    RCLCPP_INFO_STREAM(
        node->get_logger(),
        "RS485 response bytes: " << ss.str()
    );
}

);
}

```

Kao povratnu vrednost, dobijali smo nule. Povezivanjem osciloskopa ustanovili smo da jedan od dva kanala nije funkcionalan, te je proverom donglice na control box-u utvrđeno da je spržen pin.